

MODULE NAME: UNIT TEST FRAMEWORKS

EXPERIMENT TITLE: RADIO BUTTON

Problem Statement

To identify and interact with radio buttons on a webpage using Selenium WebDriver and verify the selection of the required option.

Objective

To learn how to locate, select, and verify radio buttons using Selenium WebDriver.

Tools, Software, and Concepts Used

- **Python 3.x**
- **Selenium (latest version)**
- **Google Chrome / Mozilla Firefox**
- **Selenium WebDriver**
- **VS Code / PyCharm**
- Python packages:
 - selenium
 - webdriver-manager
- Concepts: **Radio Button, Web Element Identification, `click()`, `is_selected()`, Locators**

Source Code / Implementation Steps

1. Install Selenium and WebDriver Manager.
2. Import the required Selenium modules.
3. Launch the browser using Selenium WebDriver.
4. Open the required webpage.
5. Locate the radio button using a suitable locator.
6. Click the radio button using the `click()` method.
7. Verify whether the radio button is selected using `is_selected()`.
8. Close the browser using `driver.quit()`.

Source Code :

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
from selenium.webdriver.chrome.service import Service as ChromeService
```

```
from selenium.webdriver.chrome.options import Options as ChromeOptions
```

```
from webdriver_manager.chrome import ChromeDriverManager
```

```
import time

# Chrome Options

chrome_options = ChromeOptions()

# Chrome Service

chrome_service = ChromeService(

    ChromeDriverManager().install()

)

# Start Chrome

driver = webdriver.Chrome(

    service=chrome_service,

    options=chrome_options

)

# Open website

driver.get("https://rahulshettyacademy.com/AutomationPractice/")

time.sleep(3)

# Radio 1

radio1 = driver.find_element(By.XPATH, "//input[@value='radio1']")

radio1.click()

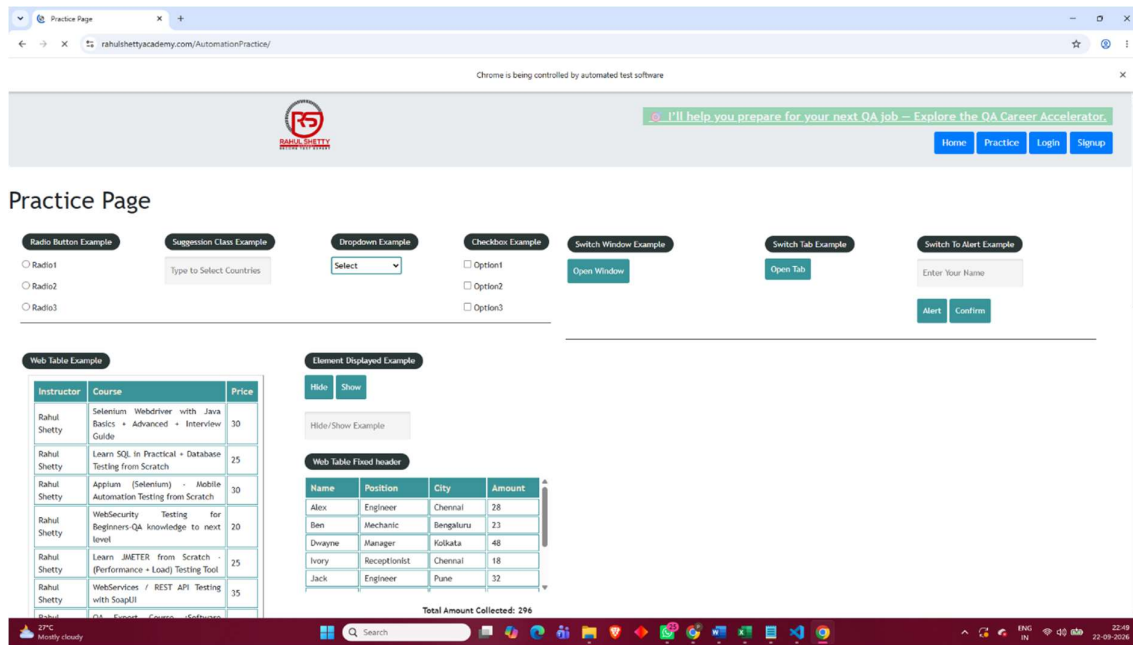
time.sleep(2)

print("Radio 1 selected:", radio1.is_selected())

time.sleep(3)

driver.quit()
```

Output with clearly labelled screenshots :



```

PS C:\Users\AASISH SHRESTHA\OneDrive\Desktop> python radiobutton.py
Radio 1 selected: True
PS C:\Users\AASISH SHRESTHA\OneDrive\Desktop>

```

Result: The required radio button was successfully identified and selected.

Observation: The `click()` method selects the radio button, while `is_selected()` verifies its selection status.

Conclusion: The experiment demonstrated how to automate and verify radio button selection using Selenium WebDriver.

EXPERIMENT TITLE: KEYBOARD ACTIONS

Problem Statement

To identify and perform keyboard actions on web elements using Selenium WebDriver, such as typing, pressing keys, copying, pasting, and using keyboard shortcuts.

Objective

To learn how to automate keyboard operations on web pages using Selenium WebDriver and the `Keys` class.

Tools, Software, and Concepts Used

- **Python 3.x**
- **Selenium (latest version)**
- **Google Chrome / Mozilla Firefox**
- **Selenium WebDriver**
- **VS Code / PyCharm**
- Python packages:
 - selenium
 - webdriver-manager
- Concepts: **Keyboard Actions, Keys class, send_keys()**, **Keyboard Shortcuts, Web Elements**

Source Code / Implementation Steps

1. Install Selenium and WebDriver Manager.
2. Import the required Selenium modules and `Keys` class.
3. Launch the browser using Selenium WebDriver.
4. Open the required webpage.
5. Locate the required input or web element.
6. Use `send_keys()` to perform keyboard actions.
7. Use keys such as `ENTER`, `TAB`, `CTRL`, `SHIFT`, and `BACKSPACE` as required.
8. Verify the performed actions and close the browser using `driver.quit()`.

Source Code :

```

from selenium import webdriver

from selenium.webdriver.common.by import By

from selenium.webdriver.chrome.service import Service as ChromeService

from selenium.webdriver.chrome.options import Options as ChromeOptions

from selenium.webdriver.common.keys import Keys

from webdriver_manager.chrome import ChromeDriverManager

import time

# Chrome Options

chrome_options = ChromeOptions()

# Chrome Service

chrome_service = ChromeService(

    ChromeDriverManager().install()

)

# Start Chrome

driver = webdriver.Chrome(

```

```

service=chrome_service,

options=chrome_options

)

# Open website

driver.get("https://rahulshettyacademy.com/AutomationPractice/")

time.sleep(3)

# Find Name field

name = driver.find_element(By.ID, "name")

# Type text

name.send_keys("Parthib Kumar Das")

time.sleep(2)

# Select all text

name.send_keys(Keys.CONTROL, "a")

time.sleep(2)

# Delete selected text

name.send_keys(Keys.BACKSPACE)

time.sleep(2)

# Type new text

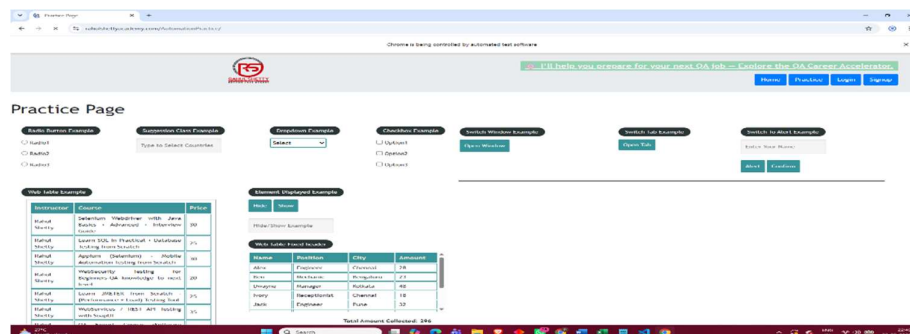
name.send_keys("Selenium Tester")

time.sleep(3)

driver.quit()

```

Output with clearly labelled screenshots :



Result: Keyboard actions were successfully performed on the web elements using Selenium.

Observation: Selenium's `Keys` class allows automation of keyboard keys and shortcuts through the `send_keys()` method.

Conclusion: The experiment demonstrated how keyboard operations can be automated efficiently using Selenium WebDriver.

EXPERIMENT TITLE: MOUSE HOVER

Problem Statement

To identify a web element and perform a mouse hover action on it using Selenium WebDriver to access or display hidden menus and options.

Objective

To learn how to perform mouse hover actions on web elements using Selenium's `ActionChains` class.

Tools, Software, and Concepts Used

- **Python 3.x**
- **Selenium (latest version)**
- **Google Chrome / Mozilla Firefox**
- **Selenium WebDriver**
- **VS Code / PyCharm**
- Python packages:
 - `selenium`
 - `webdriver-manager`
- Concepts: **Mouse Hover, ActionChains, Web Element Identification, `move_to_element()`**

Source Code / Implementation Steps

1. Install Selenium and WebDriver Manager.
2. Import Selenium and the `ActionChains` class.
3. Launch the browser using Selenium WebDriver.
4. Open the required webpage.
5. Locate the element on which the mouse hover action is required.
6. Create an `ActionChains` object.
7. Use `move_to_element()` to move the mouse over the selected element.
8. Perform the action and verify the displayed menu or option.
9. Close the browser using `driver.quit()`.

Source Code:

```
from selenium import webdriver

from selenium.webdriver.common.by import By

from selenium.webdriver.chrome.service import Service as ChromeService

from selenium.webdriver.chrome.options import Options as ChromeOptions

from selenium.webdriver.common.action_chains import ActionChains

from webdriver_manager.chrome import ChromeDriverManager

import time

# Chrome Options

chrome_options = ChromeOptions()

# Chrome Service

chrome_service = ChromeService(

    ChromeDriverManager().install())

# Start Chrome

driver = webdriver.Chrome(

    service=chrome_service,

    options=chrome_options)

# Open website

driver.get("https://rahulshettyacademy.com/AutomationPractice/")

time.sleep(3)

# Find Mouse Hover button

mouse_hover = driver.find_element(By.ID, "mousehover")

# Create ActionChains

actions = ActionChains(driver)

# Move mouse over the button

actions.move_to_element(mouse_hover).perform()

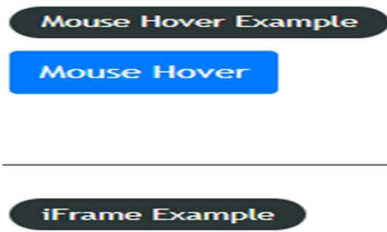
# Wait so you can see the hidden menu

time.sleep(3)

# Close browser
```

```
driver.quit()
```

Output with clearly labelled screenshots :



Result: The mouse hover action was successfully performed on the selected web element.

Observation: `ActionChains` and `move_to_element()` are used to simulate mouse movement and hover actions.

Conclusion: The experiment demonstrated how Selenium can automate mouse hover interactions on web pages.

EXPERIMENT TITLE : ALERTS

PROBLEM STATEMENT

To identify, handle, and interact with JavaScript alert pop-ups on a webpage using Selenium WebDriver.

OBJECTIVE

To learn how to handle different types of alerts such as **Simple Alert, Confirmation Alert, and Prompt Alert** using Selenium WebDriver.

TOOLS, SOFTWARE, AND CONCEPTS USED

- Python 3.x
- Selenium (latest version)
- Google Chrome
- Selenium WebDriver
- VS Code / PyCharm
- Python packages:
 - selenium
 - webdriver-manager
- Concepts: **Alerts, switch_to.alert, accept(), dismiss(), send_keys(), text**

The given practice website contains **Simple Alert, Confirmation Alert, and Prompt Alert** buttons.

SOURCE CODE / IMPLEMENTATION

```
from selenium import webdriver

from selenium.webdriver.chrome.service import Service

from selenium.webdriver.common.by import By

from webdriver_manager.chrome import ChromeDriverManager

import time

service = Service(ChromeDriverManager().install())

driver = webdriver.Chrome(service=service)

driver.get("https://testautomationpractice.blogspot.com/")

driver.maximize_window()

time.sleep(2)

# ----- Simple Alert -----

driver.find_element(By.XPATH, "//button[text()='Simple Alert']").click()

time.sleep(1)

alert = driver.switch_to.alert

print("Simple Alert:", alert.text)

alert.accept()

time.sleep(2)

# ----- Confirmation Alert -----

driver.find_element(By.XPATH, "//button[text()='Confirmation Alert']").click()

time.sleep(1)

alert = driver.switch_to.alert

print("Confirmation Alert:", alert.text)

alert.accept()

time.sleep(2)

# ----- Prompt Alert -----

driver.find_element(By.XPATH, "//button[text()='Prompt Alert']").click()

time.sleep(1)
```

```

alert = driver.switch_to.alert

print("Prompt Alert:", alert.text)

alert.send_keys("Parthib")

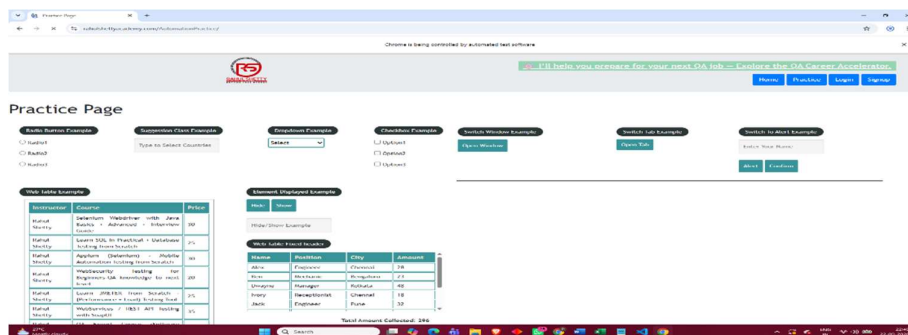
alert.accept()

time.sleep(2)

driver.quit()

```

Output with clearly labelled screenshots :



Result: Different types of alerts were successfully identified and handled using Selenium WebDriver.

Observation: Selenium uses `switch_to.alert` to interact with JavaScript alerts and provides methods such as `accept()`, `dismiss()`, and `send_keys()`.

Conclusion: The experiment demonstrated how to automate and handle Simple, Confirmation, and Prompt alerts using Selenium WebDriver.

Name: Aasish Shrestha , Enrollment No: 12023002001003 , CSE , Roll No: 33
 Institute of Engineering & Management, Kolkata
 Email ID: aasish.shrestha2023@iem.edu.in
aasishshrestha2005@gmail.com